实时数据分发中间件

SkyDDS 3.0

BatchOctetMsg（C++）

应用开发指南

2024.06

北京鼎轩科技

目　录

1 实时数据分发中间件 SkyDDS....4
1.1 DDS 简介....4
1.2 DDS 概念模型....4
1.2.1 通信域（Domain）....5
1.2.2 域参与者（DomainParticipant）....5
1.2.3 主题（Topic）....5
1.2.4 发布者（Publisher）....5
1.2.5 订阅者（Subscriber）....5
1.2.6 数据写者（DataWriter）....5
1.2.7 数据读者（DataReader）....6
1.3 DDS 核心对象及关系....6
1.3.1 核心对象和类....6
1.3.2 核心对象层级关系....6
1.3.3 Listener 监听器层级关系....7
1.4 DDS 技术架构....8
1.5 无中心、分布式部署....9
1.6 DDS 通信终端自动发现技术....9
1.6.1 RTPS 对等自动发现服务....10
1.6.2 Repo 集中自动发现服务....10
1.6.3 Redis 注册发现服务....11
2 DDS 传输推荐配置....12
2.1 tcp 传输推荐配置....12
2.2 rtps_udp 传输推荐配置（非可靠）....13
2.3 rtps_udp 传输推荐配置（可靠性）....14
3 SkyDDS 安装及使用....15
3.1 Windows 平台 SkyDDS3.0 安装....15
3.1.1 安装....15
3.1.2 设置环境变量....15
3.1.3 运行 demo 实例（examples\SatelliteBatchMsg）....18
3.2 Linux 平台 SkyDDS3.0 安装....19
3.2.1 安装....19
3.2.2 设置环境变量....20
3.2.3 运行 demo 实例（examples\SatelliteBatchMsg）....20
3.3 产品包目录结构....21
3.4 基于 SkyDDS 开发的基本依赖....22
4 BatchOctetMsg 应用开发示例....23
4.1 软件准备和环境设置....23
4.1.1 设置 DDS 环境....23
4.1.2 DDS 环境变量....23
4.2 应用开发....24
4.2.1 Satellite.idl 及生成的文件....24
4.2.2 编写 publisher 和 subscriber 代码....25
4.2.2.1 examples\SatelliteBatchMsg\publisher_Satellite.cpp....25
4.2.2.2 examples\SatelliteBatchMsg\subscriber_Satellite.cpp....29
4.2.3 编译 publisher 和 subscriber 程序....32
4.2.4 Windows 平台....32

4.2.4.1 构建 vs2015 工程 32
4.2.4.2 编译 Windows 工程 32
4.2.5 Linux 平台 33
4.2.5.1 Makefile 33
4.2.6 运行 SatelliteBatchMsg 实例 34
4.2.6.1 Windows 平台 34
4.2.6.2 Linux 平台 35

1 实时数据分发中间件 SkyDDS

1.1 DDS 简介

SkyDDS（Data Distribution Service），是北京鼎轩科技公司研发的新一代、无中心、分布式部署的通信中间件，具有如下的技术特征

1、遵循 OMG 组织颁布的分布式实时数据分发 DDSv1.4 标准规范；

2、具有以数据为中心的发布订阅（DCPS，Data-Centric Publish-Subscribe）机制、提供了基于主题 Topic 的发布/订阅模型的通信功能；

3、具有网络传输协议多样化、通信协议可插拔性（tcp/rtps_udp 等配置可选性，应用程序无需任何修改）；

4、提供分布式、匿名化终端节点间的发布/订阅型通信服务，即无须关心和提前配置发布方、订阅方的本机 IP 或主机名等位置信息；

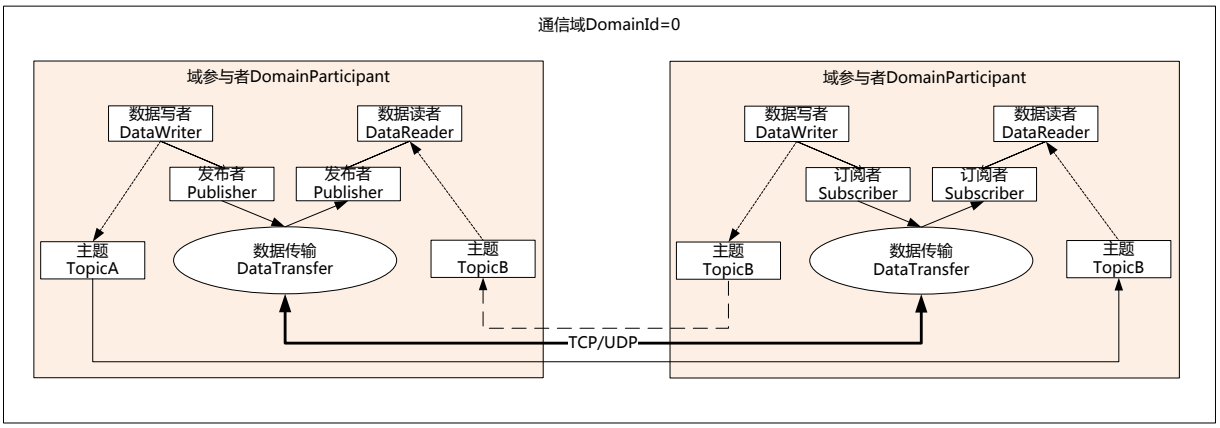
5、提供 23 种丰富的 QoS 服务质量策略（如可靠性 Reliability、活跃性 Liveness、逻辑分区 Partition、生命周期 Lifespan、消息顺序性 DestinationOrder、基于时间的消息过滤 Time_Base_Filter、发布属主权 OwnerShip、消费组 UserData 等），可根据业务需要灵活地控制数据进行有效地分发；

6、具有低延迟、强实时、高吞吐的高效传输性能；

7、具备数据分发通信的互操作性，即与其他 DDS 系统可互连、互通（仅限于基于 RTPS 对等发现的 rtps_udp 通信协议）；

是一种轻便、高效、易用的实时数据传输的通信中间件产品。

1.2 DDS 概念模型



1.2.1 通信域（Domain）

➢ 通信域是基于主题发布订阅通信（DCPS）的基本单位，定义了 DDS 独立的通信平面。其他 DDS 实体都必须属于某一个通信域；同时，DDS 实体只能与相同域的 DDS 实体进行交互。

➢ 通信域以 domainId 如 0,1,2,……,255 等数字来标示。

1.2.2 域参与者（DomainParticipant）

➢ 域参与者是一个应用程序和特定域交互的入口点。域参与者是一个供多对象如 Publisher 发布者、Subscriber 订阅者、Topic 主题等参与数据读写的工厂。

1.2.3 主题（Topic）

➢ 主题是实现发布和订阅之间的基本手段。每一个主题在域中都有一个唯一的 Name 和一个特定的发布数据 Type，每个主题数据类型可以由零或多个字段构成其主键。

➢ 当发布数据时，发布程序总是指定主题，用户通过主题请求数据。在 DCPS 中，每个实例通过一个主题可以给多个实例发布应用数据。每个实例都通过唯一的主键关联。一个发布进程可以通过一个实例及一个主键发布多个数据，每发布一次数据用一次主键。

1.2.4 发布者（Publisher）

➢ 发布者负责将 DataWriter(数据写者)已发布的数据分发给域中的所有相关订阅者。

1.2.5 订阅者（Subscriber）

➢ 订阅者接收来自发布者的数据，并将其传递给和其相连的 DataReader(数据读者)。

1.2.6 数据写者（DataWriter）

➢ DataWriter 用于发布程序将数据传送给 DDS。每个 DataWriter 都和特定的主题绑定，应用程序调用 DataWriter 特定接口来发布该主题的数据，DataWriter 实体负责数据封装及传送。

1.2.7 数据读者（DataReader）

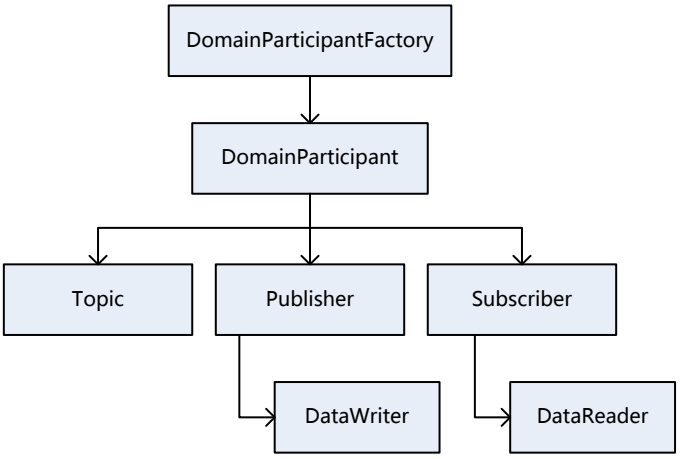
➢ DataReader 从订阅者读取数据，拆封成合适的应用数据，并且将这个数据传给应用程序。每个 DataReader 都和一个特定主题绑定，应用程序调用 DataReader 的特定数据类型接口来接收数据。

1.3 DDS 核心对象及关系

1.3.1 核心对象和类

	标识	说明
1.	DomainParticipantFactory	域参与者工厂
2.	DomainParticipant	域参与者
3.	Publisher	发布者
4.	Subscriber	订阅者
5.	Topic	主题
6.	DataWriter	数据写者
7.	DataReader	数据读者
8.	DataReaderListener	数据读者监听器基类
9.	xxxDataReaderListener	数据读者监听器实体类（用户自定义）
10.	DataWriterListener	数据写者监听器基类
11.	xxxDataWriterListener	数据写者监听器实体类（用户自定义）

1.3.2 核心对象层级关系



➢ DomainParticipantFactory 是单例对象，也是 DomainParticipant 的工厂；

➢ DomainParticipant 是某个通信域 Domain（由 domainId 指定）的域参与者，Topic、Publisher 和 Subscriber 的工厂；属于同一个通信域 Domain 的实体对象可以通讯，不同

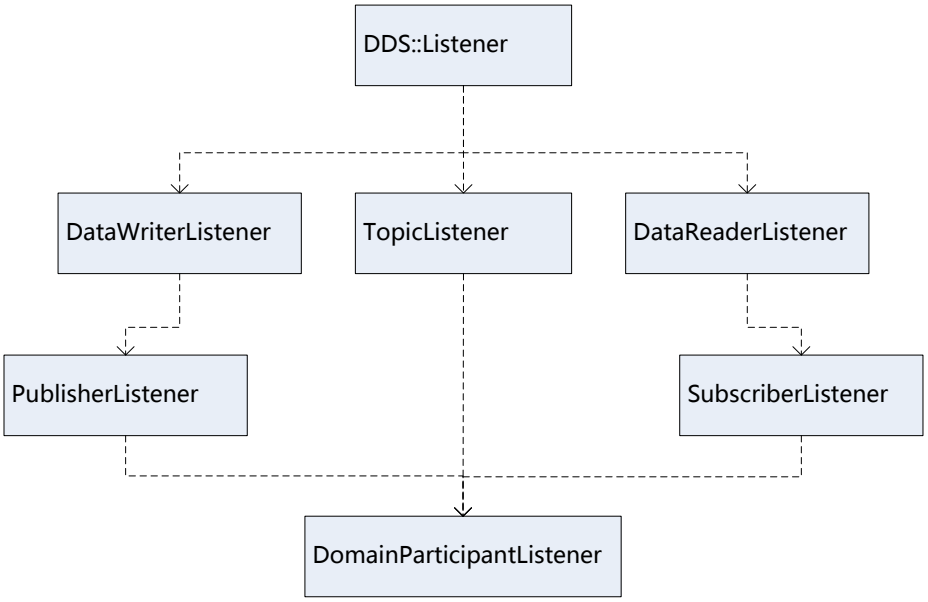
Domain 的实体对象不可以通讯；

➢ Topic 具有自己的名字（TopicName）和类型（Type），是 DataWriter 和 DataReader 间连接和通讯的逻辑线索；

➢ Publisher 是 DataWriter 的工厂，可以创建多个 DataWriter；DataWriter 只属于某一个 Publisher，并与某一个 Topic 相关联；

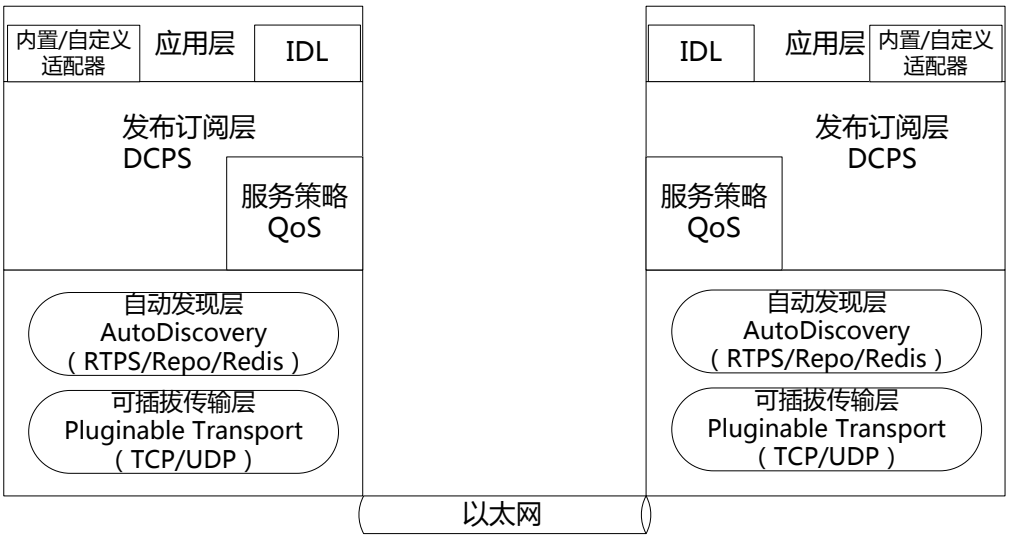
➢ Subscriber 是 DataReader 的工厂，可以创建多个 DataReader；DataReader 只属于某一个 Subscriber，并与某一个 Topic 相关联；

1.3.3 Listener 监听器层级关系



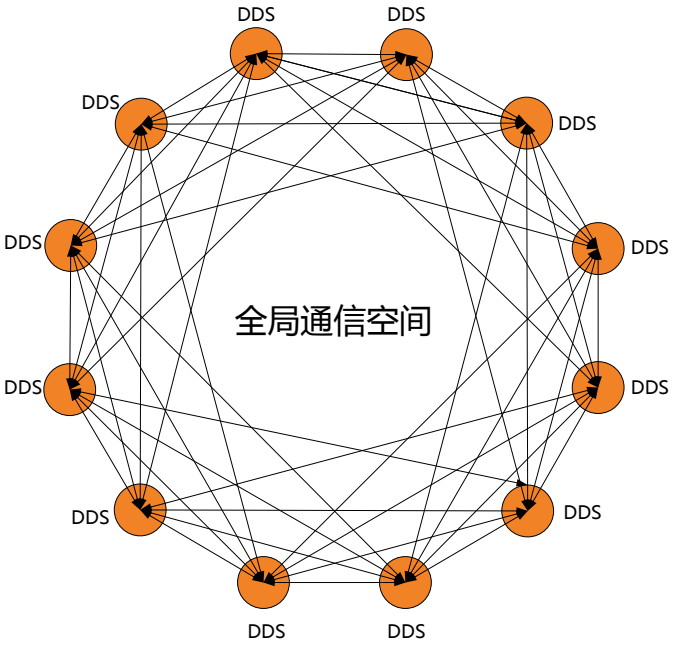
（Listener 继承关系图）

1.4 DDS 技术架构



1) DDS 应用层，可以基于 DDS-SDK 提供的接口，定义不同形式（同步、异步和触发等）的数据发布和数据订阅流程；同时，可以订阅 DDS 传输中的事件监听器，对触发的事件如主题逻辑匹配、连接删除进行响应和处理；
2) DDS 应用层中的业务消息，支持 IDL 规范数据传输消息的自定义，应用层发布结构体、类等对象，订阅方也接收到相应的结构体和类对象，具有可见即所得的效果；
3) DDS 发布-订阅层（DCPS），奠定了以数据主题为主线的数据传输模式，允许应用层根据数据的传递关系分别构建发布者和订阅者；同一个应用程序内，可以定义 1 个或多个发布者，也可以同时定义 1 个或多个订阅者，与其他应用程序内的发布者和订阅者，完成以数据为中心的发布-订阅通信。
4) DDS 发现层，具有自动发现终端的能力，自动识别出全局通信空间内的发布者和订阅者，实时动态逻辑匹配，发布者和订阅者间自动完成数据传输的网络链路搭建和自动维护；
5) DDS 传输层，具有传输协议可插拔的服务能力，提供了 TCP、UDP、Multicast 和共享内存等传输协议；可插拔传输协议的优势在于，应用层可以在配置文件指定或应用程序中动态定义的方式来自主选择，即业务数据传输的应用开发和传输设施及协议，是低耦合的；
6) DDS 数据分发服务，提供了丰富多样的数据传输服务策略 QoS。QoS 负责对数据传输质量的定义和控制，包括可靠性 Reliability、有效期 Lifespan、所有权 Ownership、消费组 UserData 等等，不同的应用业务可以根据业务特点选择 QoS 策略和参数，对传输过程和数据质量进行控制；

1.5 无中心、分布式部署



DDS 数据分发服务，提供了“全局通信空间”内的分布式数据传输服务。在“全局通信空间”内，包括本地网、相邻网、异地网、互联网、云平台网络如虚拟机或容器等网络环境，提供了基于主题的发布订阅式的数据通信服务。

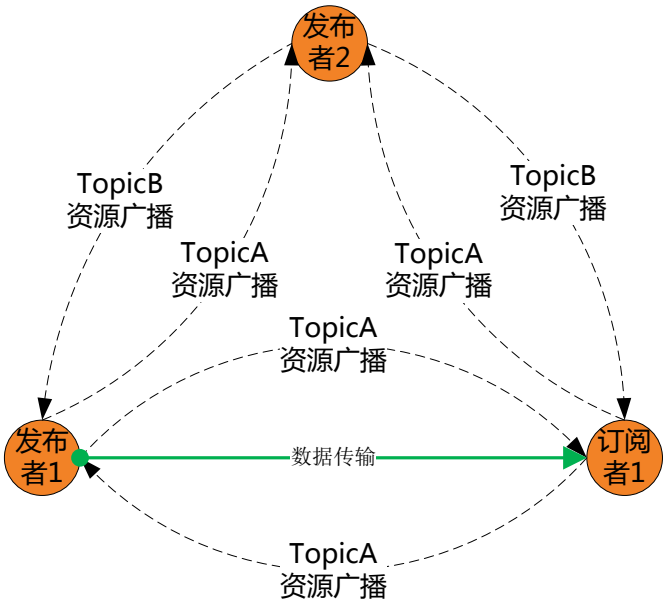
1) DDS 数据分发服务,甄别具有相同逻辑元组（通讯域 DomainId，通信主题，数据类型,实体 QoS）等特征的发布者和订阅者，提供分布式数据传输服务；
2) DDS 数据分发服务，提供了 1 对 1 发布订阅、1 对多发布订阅、多对 1 发布订阅、多对多发布订阅等多种模式的数据传输服务；
3) 基于 DDS 数据分发服务的消息，消息类型和格式具有自定义和可扩展性；数据的消息类型,是基于 IDL（Interface Description Language）规范进行定义。基于 IDL 规范定义的消息报文，具有“可见即所得”的可视化定义的特点，传输的消息结构体、类等数据对象，对发布者和订阅者都是透明可见的。

1.6 DDS 通信终端自动发现技术

DDS 数据分发服务，具有数据通信终端自动发现技术，即在全局通讯空间内，发布者动态、实时发现订阅者，并参考配置文件和代码动态建立数据传输链路，进行数据传输的技术。

DDS 数据分发服务，提供多种自动发现技术服务，同时支持用户级自定义通信终端注册发现服务。

1.6.1 RTPS 对等自动发现服务

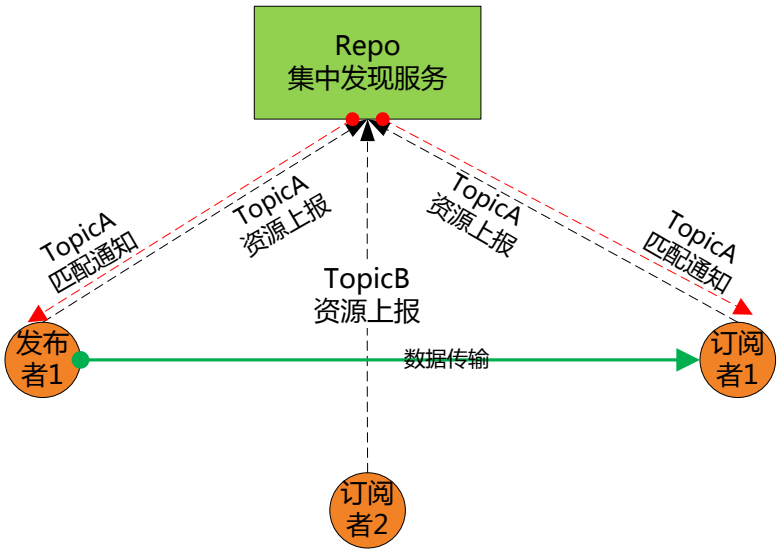


DDS 的 RTPS 对等自动发现

发布者和订阅者，通过组播通信协议，相互发布各自的通讯主题和消息类型，只有(通信域 DomainId，通讯主题名称，消息类型，QoS 策略)一致或兼容的发布者和订阅者，才能在数据主题层匹配成功，自动构建传输通道，并进行数据的传输。

发布者 1 对外组播 TopicA 的发布信息，发布者 2 对外组播 TopicB 的发布信息，订阅者 1 对外组播 TopicA 的订阅信息，经过协商和匹配，发布者 1 获取到订阅者 1 的订阅监听地址信息如 ip 和 port，根据通信协议自动构建传输通道，并将数据从发布者 1 持续传输到订阅方 1。

1.6.2 Repo 集中自动发现服务

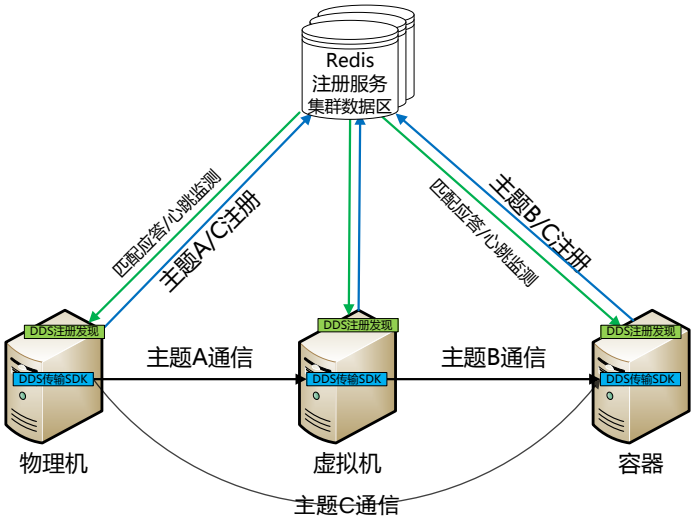


DDS 集中发现服务

Repo 集中发现服务，作为发布者和订阅者间的主题匹配的协调服务中心，分别接收来自所有发布者和订阅者的主题资源信息，协助各方的发布者和订阅者构建数据主题匹配的逻辑关系，并将匹配结果通知给发布者和订阅者；在后续的过程中，会动态、实时地将发布者和订阅者信息动态通报给对方。

发布者 1 向集中发现服务发送 TopicA 的主题发布信息，发布者 2 向集中发现服务发送 TopicB 的主题发布信息，订阅者 1 向集中发现服务发送 TopicA 的主题订阅信息，集中发现服务实时协商和匹配，将订阅者 1 的订阅监听位置信息如 ip 和 port 发送个发布者 1，发布者 1 根据通信协议自动构建传输通道，并将数据持续发布到订阅方。

1.6.3 Redis 注册发现服务



Redis 注册发现服务基于云+容器环境设计理念，以最小的网络连接、最简单的连接关系、实时的探活心跳策略，实现 DDS 通信终端主题资源的注册/发现、主题匹配关系的动态维护等。

以 Redis 集群数据库作为 DDS 各级对象的共享存储区，以 DDS 注册发现插件为前端，通过插件提供 DDS 主题资源信息的注册、关联资源拉取的动态匹配和心跳式探活，来实现 DDS 通信终端间的注册发现服务。

1) Redis 集群，作为 DDS 注册发现资源数据（主题信息、通信终端地址等）的高可用、高性能、高并发访问存储区；
   a) DDS 注册发现前端（插件化运行，SDK-API 调度和激活），运行于 DDS 的发布 pub 和 sub 的 sdk 内，主动向 redis 建立连接；
   b) 存储于 Redis 内的注册数据，都具有生命周期（如 ttl=30），超过周期后，自动销毁。
2) DDS 注册发现前端插件；

a) 主动向 redis 建立网络连接，以定时心跳方式将将主题信息（发布/订阅请求）和相关资源（订阅端监听地址和策略）注册到 redis 数据库集群存储区；

b) 从 redis 存储区，检索当前 DDS 对象所需要和关联的（domainId，topic）信息，并进行本地的主题匹配监测

   i. 新增加的 pub 和 sub 匹配信息，进行本地的主题通知，如 on_publication_match（current_count_change=1）、on_subscription_match（current_count_change=1）通知；

   ii. 已经下线的 pub 和 sub 匹配信息，进行本地的主题通知，如 on_publication_match（current_count_change=-1）、on_subscription_match（current_count_change=-1）通知；

   iii. 已经存在的 pub 和 sub 匹配信息，更新本地的主题状态时间戳，不进行任何通知，保持主题匹配状态不变。

c) 网络闪断后，异常恢复机制

   i. 网络闪断且超时后，redis 存储区自动释放该 DDS 对象存储区；

   ii. Pub 或 sub 监测不到 DDS 对象，则自动隔离和通知 DDS 对象下线，如 on_publication_match（current_count_change=-1）、on_subscription_match（current_count_change=-1）通知；

   iii. 网络恢复后，DDS 注册发现前端自动向 redis 服务器建立连接，并重新注册 DDS 对象，以实现与远程 DDS 终端的资源匹配。

Redis 注册发现服务，适用于云平台的物理机、虚拟机、容器中部署应用系统间 DDS 通信主题的自动发现和网络链路创建维护。

2 DDS 传输推荐配置

2.1 tcp 传输推荐配置

dds_redis_tcp_conf.ini	
DCPSGlobalTransportConfig=$file	
DCPSDebugLevel=4	#控制 DCPS 层 dubug 信息输出（0-10）
DCPSTransportDebugLevel=4	#控制传输日志粒度(0-10)
ORBLogFile=xxjh_publisher.log	#SkyDDS 运行日志文件
#RedisDiscovery 用 Redis 服务器集群信息	
[redis_discovery/fast_redis]	
RedisServer=RedisDiscoverySvr:6379	#Redis 集群地址 IP:Port
RedisConnectTimeout=1000	#连接超时时间(ms)

```
RedisPasswd=Cestc@2022                  #认证密码。无密码时为空
RedisHeartPeriod=5                      #Redis 资源检索心跳间隔(秒)
RedisTTL=15                             #Redis 键值的 ttl

[domain/0]
DiscoveryConfig=fast_redis              #Redis 注册发现策略
DefaultTransportConfig=redis_config0

[config/redis_config0]
transports=the_redis_transport0

[transport/the_redis_transport0]
transport_type=tcp                      #tcp 可靠传输协议
datalink_release_delay=0                #网络中断后释放延迟时间为 0（容器适用，立即释放）
conn_retry_attempts=0                   #网络中断后重试连接次数为 0（容器适用，不会重试）
#传输地址（多网卡时，需要配置）
local_address=:32100                    #绑定本地 ip 或 port；ip 为空，则选择本机第一个 ip，port 为空，则随机选择端口
public_address=10.1.2.9:                #对外公开的 ip 或 port。ip 一般为集群域名或 EIP。如果 port 为空，则和 local_addres 一致；如果 local_addres 的 port 未配置，则启用随机端口
```

2.2 rtps_udp 传输推荐配置（非可靠）

```
dds_redis_udp_conf.ini
DCPSGlobalTransportConfig=$file
DCPSDebugLevel=4                        #控制 DCPS 层 dubug 信息输出（0-10）
DCPSTransportDebugLevel=4               #控制传输日志粒度(0-10)
ORBLogFile=gnss_sh.log                  #SkyDDS 运行日志文件

#RedisDiscovery 用 Redis 服务器集群信息
[redis_discovery/fast_redis]
RedisServer=RedisDiscoverySvr:6379      #Redis 集群地址 IP:Port
RedisConnectTimeout=1000                #连接超时时间(ms)
RedisPasswd=Cestc@2022                  #认证密码。无密码时为空
RedisHeartPeriod=5                      #Redis 资源检索心跳间隔(秒)
RedisTTL=15                             #Redis 键值的 ttl
```

```
[domain/0]
DiscoveryConfig=fast_redis                #Redis 注册发现策略
DefaultTransportConfig=redis_config0

[config/redis_config0]
transports=the_redis_transport0

[transport/the_redis_transport0]
transport_type=rtps_udp                   #rtps_udp 传输协议
use_multicast=0                           #是否使用组播。0：不使用；1：使用
ttl=20                                    #传输时跨越子网的跳数
#传输地址（多网卡时，需要配置）
local_address=:32100                      #绑定本地 ip 或 port；ip 为空，则选择本机第一个 ip，port 为空，则随机选择端口
public_address=10.1.2.9:                  #对外公开的 ip 或 port。ip 一般为集群域名或 EIP。如果 port 为空，则和 local_addres 一致；如果 local_addres 的 port 未配置，则启用随机端口
```

2.3 rtps_udp 传输推荐配置（可靠性）

```
dds_redis_udp_conf.ini
DCPSGlobalTransportConfig=$file
DCPSDebugLevel=4                          #控制 DCPS 层 dubug 信息输出（0-10）
DCPSTransportDebugLevel=4                 #控制传输日志粒度(0-10)
ORBLogFile=skydds.log                     #SkyDDS 运行日志文件

#RedisDiscovery 用 Redis 服务器集群信息
[redis_discovery/fast_redis]
RedisServer=RedisDiscoverySvr:6379        #Redis 集群地址 IP:Port
RedisConnectTimeout=1000                  #连接超时时间(ms)
RedisPasswd=Cestc@2022                    #认证密码。无密码时为空
RedisHeartPeriod=5                        #Redis 资源检索心跳间隔(秒)
RedisTTL=15                               #Redis 键值的 ttl

[domain/0]
DiscoveryConfig=fast_redis                #Redis 注册发现策略
```

```
DefaultTransportConfig=redis_config0

[config/redis_config0]
transports=the_redis_transport0

[transport/the_redis_transport0]
transport_type=rtps_udp                    #rtps_udp 传输协议
use_multicast=0                            #是否使用组播。0：不使用；1：使用
ttl=20                                     #传输时跨越子网的跳数
#传输地址（多网卡时，需要配置）
#local_address=:32100              #绑定本地 ip 或 port；ip 为空，则选择本机第一个 ip，port 为空，则随机选择端口
#public_address=10.1.2.9:          #对外公开的 ip 或 port。ip 一般为集群域名或 EIP。如果 port 为空，则和 local_addres 一致；如果 local_addres 的 port 未配置，则启用随机端口

heartbeat_period=5                 #可靠性性能控制参数，DataWriter 方发送 ack/nack_req 的时间间隔（ms）
nak_response_delay=0               #可靠性性能控制参数，DataWriter 接收到 nack 后的发送 data 的延迟时间(ms)。0 表示立即发送
heartbeat_response_delay=1         #可靠性性能控制参数，DataReader 接收到 heartbeat_period 后的发送 ack/nack 应答的延迟时间(ms)
```

3 SkyDDS 安装及使用

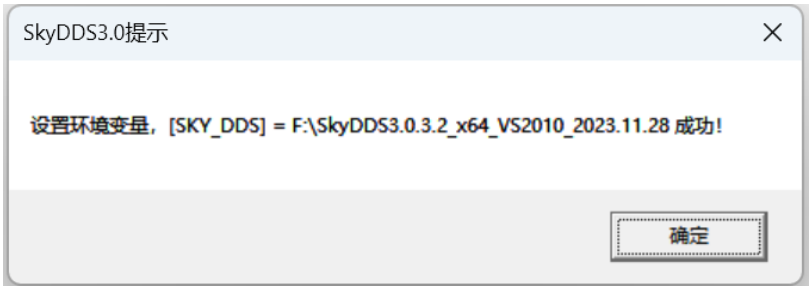
3.1 Windows 平台 SkyDDS3.0 安装

3.1.1 安装

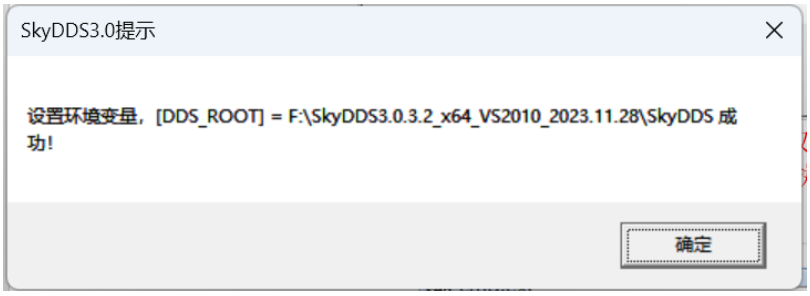
解压 SkyDDS3.0.3.2_x64_VS2010_2023.11.28.rar 到当前目录下。

3.1.2 设置环境变量

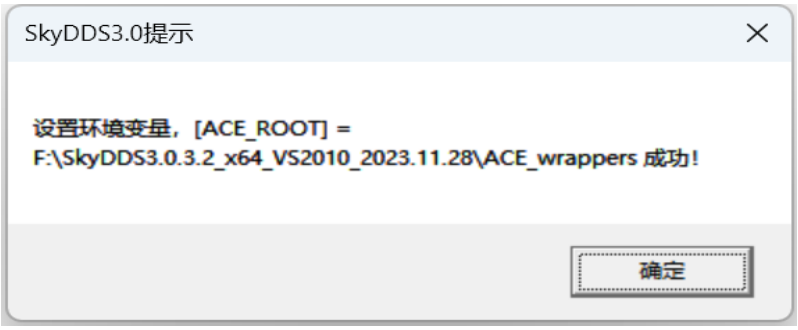
解压完毕后，需要设置 SkyDDS 运行所需的环境变量，双击 F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\dds_setp.vbs（需要 administrator 管理员权限）。



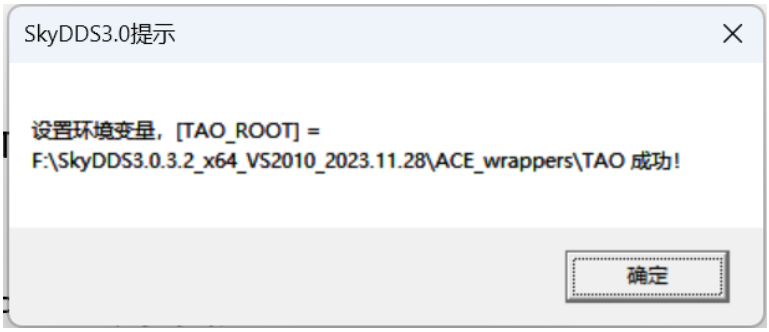
SKY_DDS 环境变量



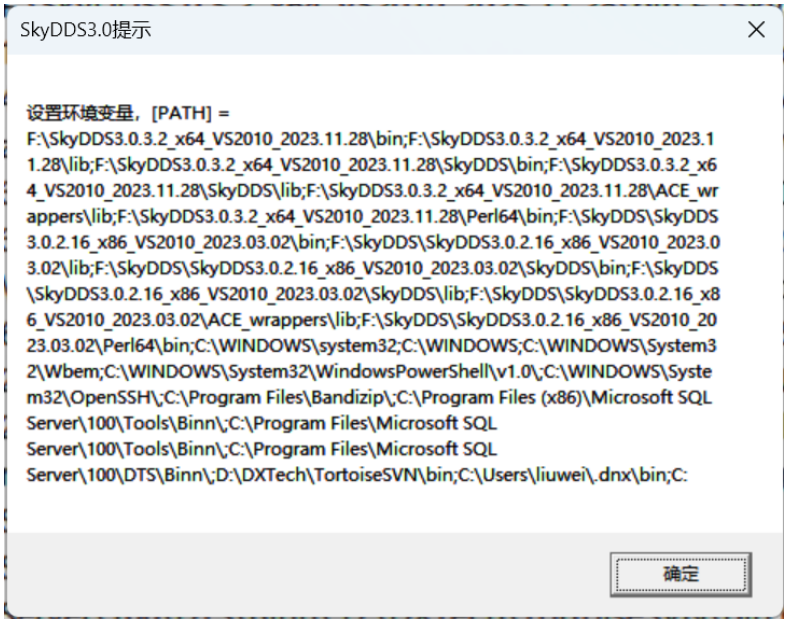
DDS_ROOT 环境变量



ACE_ROOT 环境变量

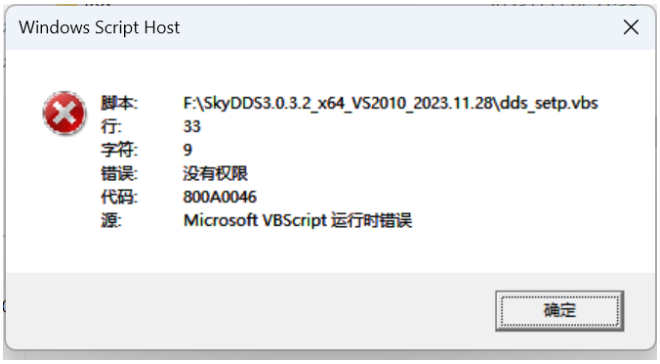


TAO_ROOT 环境变量

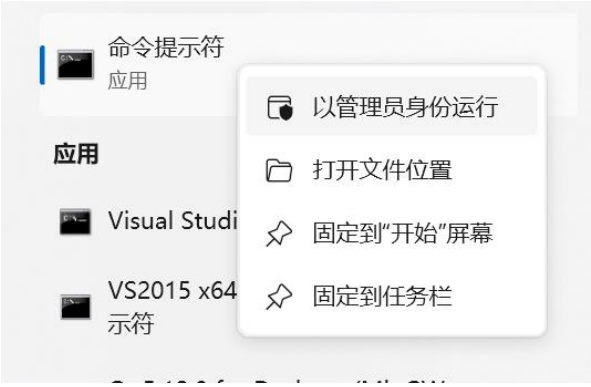


PATH 环境变量

注意：



非系统管理员运行此 vbs 文件时，可能会因为权限不足而出错。可以用“系统管理员身份”打开 cmd 命令窗口，在命令窗口中执行 dds_setp.vbs。



```
选择 管理员: 命令提示符
Microsoft Windows [版本 10.0.22621.2715]
(c) Microsoft Corporation。保留所有权利。

C:\Windows\System32>f:

F:\>cd SkyDDS3.0.3.2_x64_VS2010_2023.11.28

F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28>dds_setp.vbs

F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28>
```

3.1.3 运行 demo 实例（examples\SatelliteBatchMsg）

在安装目录下，有 start_pub.bat 和 start_sub.bat 两个批处理指令，用于运行 demo 用例。

- start_pub.bat

```
@echo off
satellite_batch_publisher.exe -topicName pdxpBatchTopic -msgNum 1000000 -dataLen 512
-batchNum 1024 -batchSize 524288 -batchDelay 1000000
-DCPSConfigFile %SKY_DDS%\etc\dds_tcp_conf.ini -ORBLogFile pdxp_publisher.log
-ORBDebugLevel 0 -DCPSDebugLevel 0 -DCPSTransportDebugLevel 0 -interval 1000 -verbose 1
```

- start_sub.bat

```
@echo off
satellite_batch_subscriber.exe -topicName pdxpBatchTopic
-DCPSConfigFile %SKY_DDS%\etc\dds_tcp_conf.ini -ORBLogFile pdxp_subscriber.log
-ORBDebugLevel 0 -DCPSDebugLevel 0 -DCPSTransportDebugLevel 0 -verbose
1-DCPSConfigFile .\etc\dds_udp_conf.ini -ORBLogFile hello_subscriber.log
```

启动两个 cmd 窗口，分别执行 start_pub.bat 和 start_sub.bat，此 demo 是基于 SkyDDS 数据发布 pub 和数据订阅 sub，执行结果如下：

- start_pub.bat

```
start_pub.bat

F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\examples\SatelliteBatchMsg>start_pub.bat

F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\examples\SatelliteBatchMsg>satellite_batch_publisher.exe -topicName pdxpBatchTopi
c -msgNum 1000000 -dataLen 512 -batchNum 1024 -batchSize 524288 -batchDelay 1000000 -DCPSConfigFile F:\SkyDDS3.0.3.2_x64
_VS2010_2023.11.28\etc\dds_tcp_conf.ini -ORBLogFile pdxp_subscriber.log -ORBDebugLevel 0 -DCPSDebugLevel 0 -DCPSTranspor
tDebugLevel 0 -interval 1000 -verbose 1
BatchOctetMsg, batchNum=1024, batchDelay=1000000ms, batchSize=524288
2023-12-04 18:20:55:101 *****SatelliteDataWriterListener::on_publication_matched***** 新增主题匹配, (topic=pdxpBatchTopi
c, type=BatchOctetMsg)
        current_count=1, current_count_change=1, total_count=1, total_count_change=1

2023-12-04 18:20:55:101, Satellite::OctetMsg wrote a sample, count=1
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
2023-12-04 18:20:56:114, Satellite::OctetMsg wrote a sample, count=2
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
2023-12-04 18:20:57:127, Satellite::OctetMsg wrote a sample, count=3
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
2023-12-04 18:20:58:139, Satellite::OctetMsg wrote a sample, count=4
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
2023-12-04 18:20:59:153, Satellite::OctetMsg wrote a sample, count=5
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
2023-12-04 18:21:00:169, Satellite::OctetMsg wrote a sample, count=6
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
```

- start_sub.bat

```
start_sub.bat

F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\examples\SatelliteBatchMsg>start_sub.bat

F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\examples\SatelliteBatchMsg>satellite_batch_subscriber.exe -topicName pdxpBatchTop
ic -DCPSConfigFile F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\etc\dds_tcp_conf.ini -ORBLogFile pdxp_subscriber.log -ORBDebug
Level 0 -DCPSDebugLevel 0 -DCPSTransportDebugLevel 0
2023-12-04 18:20:55:101, *****SatelliteDataReaderListener::on_subscription_matched***** 新增主题匹配, (topic=pdxpBatchTo
pic, type=BatchOctetMsg)
        current_count=1, current_count_change=1, total_count=1, total_count_change=1

2023-12-04 18:20:56:114, (pdxpBatchTopic, BatchOctetMsg) received a sample, batchData.length()=1
2023-12-04 18:20:56:114, BatchOctetMsg received a sample, counter=1
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
        dataBody    = [SSSS
```

```
tar -xvf R_SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28_gcc7.5.0_glibc2.27.tar.gz
```

```
skydds@marvsmart:~/DXTech$ tar -xvf R_SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28_gcc7.5.0_glibc2.27.tar.gz
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Types.hpp
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Types.hpp: 时间戳 2023-11-24 20:56:14 是未来的 3587101.210469007 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Types.ipp
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Types.ipp: 时间戳 2023-11-24 20:56:14 是未来的 3587101.208811465 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/id_map.hpp
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/id_map.hpp: 时间戳 2023-11-24 20:56:14 是未来的 3587101.20800034 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Traversal.hpp
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Traversal.hpp: 时间戳 2023-11-24 20:56:14 是未来的 3587101.206944799 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Traversal.ipp
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Traversal.ipp: 时间戳 2023-11-24 20:56:14 是未来的 3587101.206012049 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Writer.hpp
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Writer.hpp: 时间戳 2023-11-24 20:56:14 是未来的 3587101.205200924 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Writer.ipp
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema/Writer.ipp: 时间戳 2023-11-24 20:56:14 是未来的 3587101.204366465 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XSCRT/
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XMLSchema: 时间戳 2023-11-24 20:56:14 是未来的 3587101.203859549 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XSCRT/Elements.hpp
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XSCRT/Elements.hpp: 时间戳 2023-11-24 20:56:14 是未来的 3587101.194843257 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XSCRT/Parser.hpp
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XSCRT/Parser.hpp: 时间戳 2023-11-24 20:56:14 是未来的 3587101.193851882 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XSCRT/XML.hpp
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/ace/XML_Utils/XSCRT/XML.hpp: 时间戳 2023-11-24 20:56:14 是未来的 3587101.19285409 秒之后
……
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/build_scripts/jni_check.pl: 时间戳 2023-11-24 20:56:19 是未来的 3587097.56871817 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/build_scripts: 时间戳 2023-11-24 20:56:19 是未来的 3587097.568553378 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/i2jrt_ORB.h
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/i2jrt_ORB.h: 时间戳 2023-11-24 20:56:19 是未来的 3587097.565821045 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/i2jrt_TAOObject.h
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/i2jrt_TAOObject.h: 时间戳 2023-11-24 20:56:19 是未来的 3587097.565502836 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/idl2jni_BaseJavaPeer.h
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/idl2jni_BaseJavaPeer.h: 时间戳 2023-11-24 20:56:19 是未来的 3587097.565381503 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/idl2jni_jni.h
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/idl2jni_jni.h: 时间戳 2023-11-24 20:56:19 是未来的 3587097.565240045 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/idl2jni_runtime_Export.h
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/idl2jni_runtime_Export.h: 时间戳 2023-11-24 20:56:19 是未来的 3587097.565138253 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/idl2jni_runtime.h
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime/idl2jni_runtime.h: 时间戳 2023-11-24 20:56:19 是未来的 3587097.564921253 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/start_dcpsinforepo_persistence.sh
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni/runtime: 时间戳 2023-11-24 20:56:19 是未来的 3587097.564840753 秒之后
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java/idl2jni: 时间戳 2023-11-24 20:56:19 是未来的 3587097.564805461 秒之后
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/java: 时间戳 2023-11-24 20:56:19 是未来的 3587097.564767545 秒之后
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS: 时间戳 2023-11-24 20:56:19 是未来的 3587097.564711545 秒之后
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/start_dcpsinforepo_persistence.sh: 时间戳 2023-11-24 21:02:23 是未来的 3587461.56458292 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/stop_dcpsinforepo.sh
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/stop_dcpsinforepo.sh: 时间戳 2023-11-24 21:02:23 是未来的 3587461.564486378 秒之后
SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/setenv.sh
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/setenv.sh: 时间戳 2023-11-25 09:18:02 是未来的 3631600.564353086 秒之后
tar: SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28: 时间戳 2023-11-25 09:18:16 是未来的 3631614.564244586 秒之后
skydds@marvsmart:~/DXTech$
```

3.2.2 设置环境变量

进入到解压后的目录中，执行设置环境变量的脚本。

```
cd SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28
. ./setenv.sh 或 source ./setenv.sh
```

```
skydds@marvsmart:~/DXTech$ cd SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28
skydds@marvsmart:~/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28$ . ./setenv.sh
DDS_ROOT=/home/skydds/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS
ACE_ROOT=/home/skydds/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers
TAO_ROOT=/home/skydds/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/TAO
PATH=/home/skydds/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/bin:/home/skydds/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/SkyDDS/lib:/home/skyd
ds/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/bin:/home/skydds/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/ACE_wrappers/lib:/bin:/home/sk
ydds/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/bin:/home/skydds/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/Perl64/bin:/bin:.:/usr/local/sbin:/usr/lo
cal/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games
SKYDDS_NODE=SkyDDSNode1
DDS_MONITOR_PERIODIC=5
skydds@marvsmart:~/DXTech/SkyDDS3.0.3.2 Linux4.4.179 aarch64 2023.11.28$
```

注意：setenv.sh 脚本中 JAVA_HOME 环境变量的信息，需要根据实际情况进行配置。

```
export JAVA_HOME=/home/skydds/jdk1.8.0_301
```

3.2.3 运行 demo 实例（examples\SatelliteBatchMsg）

进入到 SatelliteBatchMsg 的 demo 的目录下，在两个 ssh 窗口中分别执行 run_pub.sh 和 run_sub.sh 脚本，启动 Publisher 和 Subscriber 程序。

```
cd examples\SatelliteBatchMsg
. ./ run_pub.sh
. ./ run_sub.sh
```

- run_pub.sh

```
skydds@marvsmart:~/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/examples/SatelliteBatchMsg$ . ./run_pub.sh
BatchOctetMsg, batchNum=1024, batchDelay=1000000ms, batchSize=524288
2023-10-14 10:02:43:012 *****SatelliteDataWriterListener::on_publication_matched***** [redacted], (topic=pdxpBatchTopic, type=BatchOctetMsg)
        current_count=1, current_count_change=1, total_count=1, total_count_change=1

5 seconds later, program will exit
wait for 5's, will to exit
skydds@marvsmart:~/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/examples/SatelliteBatchMsg$
```

Publisher 方运行结果

- run_sub.sh

```
skydds@marvsmart:~/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28/examples/SatelliteBatchMsg$ . ./run_sub.sh
2023-10-14 10:03:55:932, *****SatelliteDataReaderListener::on_subscription_matched***** [redacted], (topic=pdxpBatchTopic, type=BatchOctetMsg)
        current_count=1, current_count_change=1, total_count=1, total_count_change=1

2023-10-14 10:04:03:341 topic=pdxpBatchTopic timeUsed=2.379s subCount=444455 dataSize=512 SubSize=227560960B MPS=186824.30 Net=826.71(Mbps) Mem=16.62(M) CPU=11.33% match
=1 reliable=1 depth=200

2023-10-14 10:04:08:344 topic=pdxpBatchTopic timeUsed=5.51s subCount=1000000 dataSize=512 SubSize=512000000B MPS=197980.59 Net=876.07(Mbps) Mem=16.88(M) CPU=33.19% match
=1 reliable=1 depth=200

2023-10-14 10:04:11:015, *****SatelliteDataReaderListener::on_subscription_matched***** [redacted], (topic=pdxpBatchTopic, type=BatchOctetMsg)
        current_count=0, current_count_change=-1, total_count=1, total_count_change=0

2023-10-14 10:04:13:346 topic=pdxpBatchTopic timeUsed=5.51s subCount=1000000 dataSize=512 SubSize=512000000B MPS=197980.59 Net=876.07(Mbps) Mem=16.88(M) CPU=0.00% match=
0 reliable=1 depth=200

2023-10-14 10:04:18:348 topic=pdxpBatchTopic timeUsed=5.51s subCount=1000000 dataSize=512 SubSize=512000000B MPS=197980.59 Net=876.07(Mbps) Mem=16.88(M) CPU=0.20% match=
0 reliable=1 depth=200
```

Subscriber 方运行结果

3.3 产品包目录结构

目录及文件		说明
license.dat		SkyDDS 应用开发的 license。
		1)产品试用 license 授权文件（产品包自带）；
		2)正式使用 license 授权（MAC 地址和有效日期），需要向北京鼎轩科技商务渠道获取，不同计算机间 license 不能通用。
ACE_wrappers		ACE 相关资源和库（动态和静态）
etc		SkyDDS 运行配置文件
		dds_udp_conf.ini——对等发现时使用 rtps_udp 传输协议
		dds_tcp_conf.ini——对等发现时使用 tcp 传输协议
		dds_repo_udp_conf.ini——集中发现服务时使用 rtps_udp 传输协议
		dds_repo_tcp_conf.ini——集中发现服务时使用 tcp 传输协议
bin		Demo 程序的编译结果，如.dll 和.exe 等
examples	HelloMsg	Hello 消息发布订阅 demo
	HelloMsg-Java	Hello 消息发布订阅 demo
	OceanMsg	OceanMsg 消息发布订阅 demo
	OceanMsg-Java	OceanMsg 消息发布订阅 demo
	simpleHelloMsg	基于 DDSWrapper 开发的 publish 和 Subscribe 程序

	SatelliteBatchMsg	SatelliteBatchMsg 消息发布订阅 demo
	SatelliteBatchMsg_Java	SatelliteBatchMsg 消息发布订阅 demo
	simpleSatelliteBatchMsg_Java_64	基于 DDSWrapper 开发的 SatelliteBatchMsg 发布订阅程序
	netTest	SkyDDS 裸网络传输测试 demo
	DDS_Latency	时延测试 demo
	MultiHelloPerf	性能测试 demo
SkyDDS	bin	SkyDDS 相关工具，如 skydds_idl、DCPSInfoRepo 服务等
	lib	SkyDDS 相关资源和库（动态和静态）
Log		SkyDDS 运行中日志文件
其他	setenv.sh	Unix/Linux 平台用脚本
	dds_setp.bat	Windows 平台用脚本
	dds_setp.vbs	

3.4 基于 SkyDDS 开发的基本依赖

资源类型		依赖列表	备注
库资源	库目录	$(ACE_ROOT)\lib	
		$(DDS_ROOT)\lib	
	静态库	ACE.lib	
		TAO.lib	
		TAO_AnyTypeCode.lib	
		TAO_PortableServer.lib	
		SkyDDS_Dcps.lib	
	动态库	ACE.dll/.so	
		TAO.dll/.so	
		TAO_AnyTypeCode.dll/.so	
		TAO_PortableServer.dll/.so	
		SkyDDS_Dcps.dll/.so	
头文件资源	头文件所属目录	$(ACE_ROOT)	
		$(TAO_ROOT)	
		$(DDS_ROOT)	
		$(TAO_ROOT)\orbsvcs	
	头文件	#include <dds/DCPS/Service_Participant.h>	XXX 需要替换为实际的数据类型
		#include <dds/DCPS/Marked_Default_Qos.h>	
		#include <dds/DCPS/PublisherImpl.h>	
		#include <ace/streams.h>	
		#include <orbsvcs/Time_Utilities.h>	
		#include "dds/DCPS/StaticIncludes.h"	
		#include "SatelliteTypeSupportImpl.h"	

4 BatchOctetMsg 应用开发示例

4.1 软件准备和环境设置

4.1.1 设置 DDS 环境

在 Windows 平台和 Linux 平台安装好 SkyDDS3.0.x.x 后，首先需要设置环境变量：

1) Windows 平台，设置 DDS 用环境变量的两种方法（推荐方法 1）

   a) 永久性生效设置方法：切换到 F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28 目录下，以超级管理员角色运行 dds_setp.vbs，分别设置 SKY_DDS、DDS_ROOT、ACE_ROOT、TAO_ROOT 和 PATH 环境变量；

   b) 一次性生效设置方法：新打开 cmd，切换到 F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28 目录下，执行 dds_setp.bat 脚本，则在本 cmd 中设置 DDS 用环境变量，退出后无效；

2) Linux 平台，设置 DDS 用环境变量的两种方法（推荐方法 1）

   a) 永久性生效设置方法：在/etc/profile 或~/.profile 中增加，

```
cd ~/DXTech/SkyDDS3.0.3.2_Linux4.4.179_aarch64_2023.11.28
source ./setenv.sh
cd -
```

   执行 source /etc/profile 或 source .profile 或系统重启或 ssh 重新登录后，DDS 用环境变量生效。

   b) 一次性生效设置方法：每次启动一个新的 ssh 会话后，在 SkyDDS 安装目录下运行 source ./setenv.sh 或 . ./setenv.sh，分别设置 SKY_DDS、DDS_ROOT、ACE_ROOT、TAO_ROOT、PATH、CLASSPATH 和 LD_LIBRARY_PATH 环境变量。

4.1.2 DDS 环境变量

序号	变量名称	说明
1	SKY_DDS	License.dat 所在目录
2	DDS_ROOT\bin 和 DDS_ROOT\lib ACE_ROOT\bin 和 ACE_ROOT\lib	SkyDDS 用可执行程序和动态库（.dll 或.so）
3	PATH	SkyDDS 用可执行程序和动态库（.dll 或.so） 其他系统命令所在目录

4	CLASSPATH	Jar 包所在目录
5	LD_LIBRARY_PATH	Linux 平台，.so 的所在目录

4.2 应用开发

以 examples\SatelliteBatchMsg 为例进行开发。

4.2.1 Satellite.idl 及生成的文件

在产品包的 examples\SatelliteBatchMsg 目录下，已经具备了由消息 IDL 文件 Satellite.idl 生成的头文件和动态库文件，无需再次生成：

1) 动态库符号的输出头文件 SatelliteCommon_Export.h，使用 perl %ACE_ROOT%\bin\generate_export_file.pl SatelliteCommon > SatelliteCommon_Export.h 生成；
2) SatelliteTypeSupportImpl.h，由 Satellite.idl 生成；
3) libSatelliteCommon.so 或 SatelliteCommon.dll，由 Satellite.idl 生成；

➢ Satellite.idl

```
module Satellite
{
    //不限长字节序列
    typedef sequence<octet> OctetSeq;

    #pragma DCPS_DATA_TYPE "Satellite::OctetMsg"    //二进制不限长码流消息
    struct OctetMsg {
        long dataLen;                               //消息或码流数据长度
        OctetSeq dataBody;                          //消息或码流数据内容
    };

    typedef sequence<OctetMsg> SeqOctetMsg;
    #pragma DCPS_DATA_TYPE "Satellite::BatchOctetMsg"   //二进制不限长码流消息
    struct BatchOctetMsg {
        SeqOctetMsg batchData;
    };
};
```

4.2.2 编写 publisher 和 subscriber 代码

4.2.2.1 examples\SatelliteBatchMsg\publisher_Satellite.cpp

```
    DDS::DomainParticipant  * domain;
    DDS::Publisher          * publisher;
    DDS::Topic              * topic;
    DDS::DataWriter         * dw;
    DDS::ReturnCode_t       retval;

……
    //DDS 服务初始化
    DDS::DomainParticipantFactory *dpf = TheParticipantFactoryWithArgs(argc, argv);
    if (dpf == NULL) {
        printf("ERROR initializing domainParticipantFactory, please see dds log in (%s/log)
directory.\n", getenv("SKY_DDS"));
        return -1;
    }

    //创建 Satellite::BatchOctetMsg 应用实例
    DDSWrapper* sky_dds = new DDSWrapper(domainId);

    int ddsMsgNumInQue = 100;
    //定义 DataWriterQos
    DDS::DataWriterQos dw_qos = DATAWRITER_QOS_DEFAULT;
    if (iReliable) {
        dw_qos.liveliness.kind = DDS::AUTOMATIC_LIVELINESS_QOS;
    //网络活跃性（自动心跳）

        dw_qos.durability.kind = DDS::TRANSIENT_LOCAL_DURABILITY_QOS;
    //消息持久性
        dw_qos.durability_service.history_kind = DDS::KEEP_LAST_HISTORY_QOS;
        dw_qos.durability_service.history_depth = ddsMsgNumInQue;

        dw_qos.reliability.kind = DDS::RELIABLE_RELIABILITY_QOS;
    //可靠性
        dw_qos.reliability.max_blocking_time.sec = 1;
        dw_qos.reliability.max_blocking_time.nanosec = 0;

        dw_qos.history.kind = DDS::KEEP_LAST_HISTORY_QOS;
    //历史缓存，断点续传
        dw_qos.history.depth = ddsMsgNumInQue;

        dw_qos.resource_limits.max_samples = ddsMsgNumInQue;
```

```
    //资源上限
        dw_qos.resource_limits.max_samples_per_instance = ddsMsgNumInQue;
    }

    //创建数据写者，主题名称，数据类型——需要和订阅方一致
    //消息的结构类型：Satellite::BatchOctetMsg
    std::string error = "";
    //如不需要发布方回调监视器，则设置为NULL
    SatelliteDataWriterListener *dwListener = new SatelliteDataWriterListener;
    if((!sky_dds->CreateDataWriter<Satellite::BatchOctetMsg>(topicName, typeName, error,
dwListener, dw_qos)))
    {
        printf("ERROR CreateDataReader.\n");
        return -1;
    }
    //创建批量发布DataWrite_Batch对象(单体消息类型：OctetMsg，批量消息类型：BatchOctetMsg)
    Satellite::BatchOctetMsgDataWriter*                    sm_dw                    =
Satellite::BatchOctetMsgDataWriter::_narrow(sky_dds->getDataWriter(topicName));
    DDS::DataWrite_Batch<Satellite::BatchOctetMsgTypeSupportImpl,Satellite::OctetMsg>*
dw_batch                                =                                    new
DDS::DataWrite_Batch<Satellite::BatchOctetMsgTypeSupportImpl,Satellite::OctetMsg>(sm_dw);

    //设置批量发布的属性（批量消息数量，批量发布延迟时间，批量消息总大小）
    dw_batch->setBatchParam(batchNum,batchDelay,batchSize);
    printf("BatchOctetMsg,      batchNum=%d,      batchDelay=%dms,      batchSize=%d\n",
batchNum,batchDelay,batchSize);

    if(1)// 阻塞，直到有subscriber到达
    {
        // Block until Subscriber is available
        DDS::StatusCondition_var                      condition                    =
sky_dds->getDataWriter(topicName)->get_statuscondition();
        condition->set_enabled_statuses(DDS::PUBLICATION_MATCHED_STATUS);

        DDS::WaitSet_var ws = new DDS::WaitSet;
        ws->attach_condition(condition);
        DDS::PublicationMatchedStatus matches;
        while (true) {
          if
(sky_dds->getDataWriter(topicName)->get_publication_matched_status(matches) != ::DDS::RET
CODE_OK) {

printf("dw->get_publication_matched_status(matches) != ::DDS::RETCODE_OK\n");
              fflush(stdout);
          }
          //printf("current_count:%d\n",matches.current_count);
```

```
            if (matches.current_count >= subNum) {
                break;
            }

            DDS::ConditionSeq conditions;
            DDS::Duration_t timeout = { 60, 0 };
            int wait_ret = ws->wait(conditions, timeout);
            if (wait_ret != DDS::RETCODE_OK) {
                printf("ws->wait(conditions, timeout) fail, ret=%d\n", wait_ret);
                fflush(stdout);
            }
        }
        ws->detach_condition(condition);
    }

    //构建单体消息数据
    Satellite::OctetMsg msg;

    //消息内容
    msg.dataLen = tmpLen;
    msg.dataBody.length(msg.dataLen);                                    //分配数据空间,由
应用自行维护（分配和释放）
    memset((void*)&msg.dataBody[0], 'A' + (tmpLen % 26), tmpLen);

    //压缩流程
    if(compress){
        //分配与原始数据同等大小的内存区
        char *srcData = new char[tmpLen];
        char *compressedData = new char[tmpLen];
        memset(srcData, 'A' + (tmpLen % 26), tmpLen);

        //压缩
        int compressedLen = tmpLen;
        if(compress) LZ4_compress((const char*)srcData, compressedData, tmpLen);

        //msg.dataLen 保持不变(压缩前的长度)
        //dataBody 内存区数据复制
        msg.dataBody.length(compressedLen);
        if(compress)
            memcpy((void*)&msg.dataBody[0], compressedData, compressedLen);
        else
            memcpy((void*)&msg.dataBody[0], srcData, tmpLen);

        //输出
        fprintf(stderr, "originLen=%d, compressedLen=%d\n", tmpLen, compressedLen);
        delete compressedData;delete srcData;
```

```
    }

    fprintf(stderr, "publisher_Satellite domainid=%d, topicName=%s, msgNum=%d, dataLen=%d, (batchNum=%d, BatchSize=%d, BatchDelay=%dus), iReliable=%d, depth=%d, compress=%d, verbose=%d\n",
        domainId,
        topicName, msgNum, dataLen, batchNum, batchSize, batchDelay,
        iReliable, depth,
        compress,verbose);
    for (int i = 0; i < msgNum; i++)
    {
        //计算随机长度
        if (len_random_flag) {
            struct timeb T1;
            ftime(&T1);
            srand(T1.millitm);                          //获得毫秒级随机数
            tmpLen = rand() % dataLen + 1;              //生成一个随机数长度(1-len)

            //消息内容
            msg.dataLen = tmpLen;
            msg.dataBody.length(msg.dataLen);                                   //分配数据空间，由应用自行维护（分配和释放）
            memset((void*)&msg.dataBody[0], 'A' + (tmpLen % 26), tmpLen);
        }
        totalDataSize += tmpLen;

        //发送数据
        //调用 batch 方法进行发送
        DDS::ReturnCode_t ret = dw_batch->write_batch(msg);
        if ( ret != DDS::RETCODE_OK)
        {
            printf("ERROR writing Satellite::OctetMsg sample, error: %s\n", error.c_str());
            i--;
            sSLEEP(1000);
            continue;
        }
        counter++;

        //DDSWrapper 应该提供获取主题和类型的方法
        if (verbose) {
            ftime(&dds_startTime);  printf("%s, Satellite::OctetMsg wrote a sample, count=%d\n", GetCurMillTimeStr(dds_startTime), i + 1);
            cout << "   topic       = " << topicName << ", type = " << typeName << endl
                << "   dataLen     = " << msg.dataLen << endl;
                //<< "  dataBody    = [" << (char*)&msg.dataBody[0] << "]" << endl << endl;
            fflush(stdout);
```

```
        }
        //else if (0 == counter % 50000)
        //  fprintf(stderr,     "Satellite::OctetMsg     wrote     samples,     count=%d,
totalDataSize=%d\n", counter, totalDataSize);

        if(interval>0)
            mSLEEP(interval);
    }

    /* Cleanup */
    dw_batch->close();

    printf("5 seconds later, program will exit\n");
    //listener 置为空并删除
    sky_dds->getDataWriter(topicName)->set_listener(NULL, ::SkyDDS::DCPS::DEFAULT_STATUS_
MASK);
    delete dwListener;

    //5 秒后退出
    printf("wait for %d's, will to exit\n",5);
    mSLEEP(5000);

    //清理 SkyDDS 应用实例
    sky_dds->CleanUP();
    delete sky_dds;

    //DDS 服务关闭
    TheServiceParticipant->shutdown();

    return 0;
}
```

4.2.2.2 examples\SatelliteBatchMsg\subscriber_Satellite.cpp

```
int main(int argc, char * argv[])
{
    int domainId = 0;
    if(argc>=3)
    {
        for(int i=0;i<=argc-2;i++)
        {
            //主题
            if(0==(strcmp(argv[i],"-topicName")))
            {
```

```
                topicName=argv[i+1];
            }
            //信息提示
            else if (0 == (strcmp(argv[i], "-verbose")))
            {
                verbose = atoi(argv[i + 1]);
            }
            else if (0 == (strcmp(argv[i], "-compress")))
            {
                compress = atoi(argv[i + 1]);
            }
            else if (0 == (strcmp(argv[i], "-domainId")))
            {
                domainId = atoi(argv[i + 1]);
            }
            else if (0 == (strcmp(argv[i], "-reliable")))
            {
                iReliable = atoi(argv[i + 1]);
            }
        }
    }

    //DDS 服务初始化
    DDS::DomainParticipantFactory *dpf = TheParticipantFactoryWithArgs(argc, argv);
    if ( dpf == NULL )
    {
        printf("ERROR initializing domainParticipantFactory.\n");
        return -1;
    }

    //创建 Satellite::BatchOctetMsg 应用实例
    DDSWrapper* sky_dds = new DDSWrapper(domainId);

    int msgNumInQue = 30;
    //定义 DataWriterQos
    DDS::DataReaderQos dr_qos = DATAREADER_QOS_DEFAULT;
    if(iReliable){
        dr_qos.liveliness.kind = DDS::AUTOMATIC_LIVELINESS_QOS;                //网络活
跃性（自动心跳）

        dr_qos.durability.kind = DDS::TRANSIENT_LOCAL_DURABILITY_QOS;          //消息持
久性

        dr_qos.reliability.kind = DDS::RELIABLE_RELIABILITY_QOS;               //可靠性
        dr_qos.reliability.max_blocking_time.sec = 1;
        dr_qos.reliability.max_blocking_time.nanosec = 0;
```

```
        dr_qos.history.kind = DDS::KEEP_LAST_HISTORY_QOS;                        //历史缓
存，断点续传
        dr_qos.history.depth = msgNumInQue;

        dr_qos.resource_limits.max_samples = msgNumInQue;                        //资源上
限
        dr_qos.resource_limits.max_samples_per_instance = msgNumInQue;
    }

    //创建数据读者
    std::string error = "";
    SatelliteDataReaderListener *drListener = new SatelliteDataReaderListener;
    //主题名称,数据类型——需要和发布方一致
    if((!sky_dds->CreateDataReader<Satellite::BatchOctetMsg>(topicName,  typeName,  error,
drListener, dr_qos)))
    {
        printf("ERROR CreateDataReader.\n");
        return -1;
    }

    ScreenPrint[topicName].sm_dr=
Satellite::BatchOctetMsgDataReader::_narrow(sky_dds->getDataReader(topicName));

    //屏幕输出线程
#if defined(WIN32) || defined(WIN64) ||defined(_WIN32_WCE)
    DWORD thid1 = 0;
    HANDLE
th_handle1=CreateThread(NULL,0,(LPTHREAD_START_ROUTINE)PrintScreenFunc,0,0,&thid1);
#else
    pthread_t thr2;
    int ret2=pthread_create(&thr2,NULL,PrintScreenFunc,NULL);
#endif

    while ( !all_done )
        SLEEP(3);

    //listener 置为空并删除
    sky_dds->getDataReader(topicName)->set_listener(NULL, ::SkyDDS::DCPS::DEFAULT_STATUS_
MASK);
    delete drListener;

    //清理 SkyDDS 应用实例
    sky_dds->CleanUP();
    delete sky_dds;
```

```
    //DDS 服务关闭
    TheServiceParticipant->shutdown();

    return 0;
}
```

4.2.3 编译 publisher 和 subscriber 程序

4.2.4 Windows 平台

4.2.4.1 构建 vs2015 工程

参考如下的 vs2015 工程，构建 publisher 和 Subscriber 工程

- Satellite_BatchSubscriber_vs2015.vcxproj
- Satellite_BatchPublisher_vs2015.vcxproj

选择【x64】-【Release】进行编译。

需要增加 Include 路径	$(ACE_ROOT);$(TAO_ROOT);$(DDS_ROOT);$(TAO_ROOT)\orbsvcs;
需要增加 Lib 库名称	ACE.lib;TAO.lib;TAO_AnyTypeCode.lib;TAO_PortableServer.lib;SkyDDS_Dcps.lib; SkyDDS_Tcp.lib; SkyDDS_Rtps_Udp.lib;SatelliteCommon.lib;
需要增加 Lib 库路径	$(ACE_ROOT)\lib;$(DDS_ROOT)\lib;

4.2.4.2 编译 Windows 工程

使用 Visual C++ 2010，Satellite_vs2015.sln 解决方案，能看到在工作区里装载了 Satellite_Common、Satellite_BatchPublisher 和 Satellite_BatchSubscriber 三个子工程。

选择 Release x64，进行编译：

1）首先，编译 Satellite_Common，产生 Satellite_Common.dll 和 Satellite_Common.lib 库文件；

2）再分别编译 Satellite_BatchPublisher 和 Satellite_BatchSubscriber 工程，分别产生了 Satellite_BatchPublisher.exe 和 Satellite_BatchSubscriber.exe；

将产生的 Satellite_BatchPublisher.exe 和 Satellite_BatchSubscriber.exe，拷贝到 examples\SatelliteBatchMsg 路径。

4.2.5 Linux 平台

4.2.5.1 Makefile

编译 publisher_Satellite 的脚本
g++ -fPIC -O -D_REENTRANT -I$ACE_ROOT -I$TAO_ROOT -I$DDS_ROOT -I$TAO_ROOT/orbsvcs -c -o "publisher_Satellite.o" publisher_Satellite.cpp g++ -fPIC -O -D_REENTRANT -I$ACE_ROOT -I$TAO_ROOT -I$DDS_ROOT -I$TAO_ROOT/orbsvcs -c -o "Satellite_DataWriterListenerImpl.o" Satellite_DataWriterListenerImpl.cpp g++ -fPIC -O -D_REENTRANT -I$ACE_ROOT -I$TAO_ROOT -I$DDS_ROOT -I$TAO_ROOT/orbsvcs -c -o "DDSWrapper.o" DDSWrapper.cpp gcc -fPIC -O -D_REENTRANT -I$ACE_ROOT -I$TAO_ROOT -I$DDS_ROOT -I$TAO_ROOT/orbsvcs -c -o "lz4.o" lz4.c g++ -fPIC -O -D_REENTRANT -I$ACE_ROOT -I$TAO_ROOT -I$DDS_ROOT -I$TAO_ROOT/orbsvcs -L"." -L$ACE_ROOT/lib -L$DDS_ROOT/lib publisher_Satellite.o Satellite_DataWriterListenerImpl.o DDSWrapper.o lz4.o -l"SatelliteCommon" -l"SkyDDS_Tcp" -l"SkyDDS_InfoRepoDiscovery" -l"SkyDDS_Dcps" -l"TAO_PortableServer" -l"TAO_AnyTypeCode" -l"TAO" -l"ACE" -ldl -lpthread -o "satellite_batch_publisher"
编译 subscriber_Satellite 的脚本
g++ -fPIC -O -D_REENTRANT -I$ACE_ROOT -I$TAO_ROOT -I$DDS_ROOT -I$TAO_ROOT/orbsvcs -c -o "subscriber_Satellite.o" subscriber_Satellite.cpp g++ -fPIC -O -D_REENTRANT -I$ACE_ROOT -I$TAO_ROOT -I$DDS_ROOT -I$TAO_ROOT/orbsvcs -c -o "Satellite_DataReaderListenerImpl.o" Satellite_DataReaderListenerImpl.cpp g++ -fPIC -O -D_REENTRANT -I$ACE_ROOT -I$TAO_ROOT -I$DDS_ROOT -I$TAO_ROOT/orbsvcs -c -o "DDSWrapper.o" DDSWrapper.cpp gcc -fPIC -O -D_REENTRANT -I$ACE_ROOT -I$TAO_ROOT -I$DDS_ROOT -I$TAO_ROOT/orbsvcs -c -o "lz4.o" lz4.c g++ -fPIC -O -D_REENTRANT -I$ACE_ROOT -I$TAO_ROOT -I$DDS_ROOT -I$TAO_ROOT/orbsvcs -L"." -L$ACE_ROOT/lib -L$DDS_ROOT/lib subscriber_Satellite.o Satellite_DataReaderListenerImpl.o DDSWrapper.o lz4.o -l"SatelliteCommon" -l"SkyDDS_Tcp" -l"SkyDDS_InfoRepoDiscovery" -l"SkyDDS_Dcps" -l"TAO_PortableServer" -l"TAO_AnyTypeCode" -l"TAO" -l"ACE" -ldl -lpthread -o "satellite_batch_subscriber"

执行后，分别产生 satellite_batch_subscriber 和 satellite_batch_publisher 可执行程序。

4.2.6 运行 SatelliteBatchMsg 实例

本次，我们将运行一个发布 Satellite_BatchPublisher 程序，一个或多个订阅 Satellite_BatchSubscriber 程序，实现 1 个 publisher 发布、多个 subscriber 订阅的示例。

4.2.6.1 Windows 平台

在 SatelliteBatchMsg 的 demo 目录下，执行分别执行脚本 start_pub.bat 和 start_sub.bat。

- start_pub.bat

```
satellite_batch_publisher.exe -topicName pdxpBatchTopic -msgNum 1000000 -dataLen 512 -batchNum 1024 -batchSize 524288 -batchDelay 1000000 -DCPSConfigFile %SKY_DDS%\etc\dds_tcp_conf.ini -ORBLogFile pdxp_subscriber.log -ORBDebugLevel 0 -DCPSDebugLevel 0 -DCPSTransportDebugLevel 0 -interval 1000 -verbose 1
```

- start_sub.bat

```
satellite_batch_subscriber.exe -topicName pdxpBatchTopic -DCPSConfigFile %SKY_DDS%\etc\dds_tcp_conf.ini -ORBLogFile pdxp_subscriber.log -ORBDebugLevel 0 -DCPSDebugLevel 0 -DCPSTransportDebugLevel 0
```

执行结果如下：

- start_pub.bat

```
start_pub.bat

F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\examples\SatelliteBatchMsg>start_pub.bat

F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\examples\SatelliteBatchMsg>satellite_batch_publisher.exe -topicName pdxpBatchTopic -msgNum 1000000 -dataLen 512 -batchNum 1024 -batchSize 524288 -batchDelay 1000000 -DCPSConfigFile F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\etc\dds_tcp_conf.ini -ORBLogFile pdxp_subscriber.log -ORBDebugLevel 0 -DCPSDebugLevel 0 -DCPSTransportDebugLevel 0 -interval 1000 -verbose 1
BatchOctetMsg, batchNum=1024, batchDelay=1000000ms, batchSize=524288
2023-12-04 18:20:55:101 *****SatelliteDataWriterListener::on_publication_matched***** 新增主题匹配, (topic=pdxpBatchTopic, type=BatchOctetMsg)
        current_count=1, current_count_change=1, total_count=1, total_count_change=1

2023-12-04 18:20:55:101, Satellite::OctetMsg wrote a sample, count=1
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
2023-12-04 18:20:56:114, Satellite::OctetMsg wrote a sample, count=2
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
2023-12-04 18:20:57:127, Satellite::OctetMsg wrote a sample, count=3
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
2023-12-04 18:20:58:139, Satellite::OctetMsg wrote a sample, count=4
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
2023-12-04 18:20:59:153, Satellite::OctetMsg wrote a sample, count=5
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
2023-12-04 18:21:00:169, Satellite::OctetMsg wrote a sample, count=6
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
```

satellite_batch_publisher 运行结果

➢ start_sub.bat

```
start_sub.bat

F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\examples\SatelliteBatchMsg>start_sub.bat

F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\examples\SatelliteBatchMsg>satellite_batch_subscriber.exe -topicName pdxpBatchTop
ic -DCPSConfigFile F:\SkyDDS3.0.3.2_x64_VS2010_2023.11.28\etc\dds_tcp_conf.ini -ORBLogFile pdxp_subscriber.log -ORBDebug
Level 0 -DCPSDebugLevel 0 -DCPSTransportDebugLevel 0
2023-12-04 18:20:55:101, *****SatelliteDataReaderListener::on_subscription_matched***** 新增主题匹配, (topic=pdxpBatchTo
pic, type=BatchOctetMsg)
        current_count=1, current_count_change=1, total_count=1, total_count_change=1

2023-12-04 18:20:56:114, (pdxpBatchTopic, BatchOctetMsg) received a sample, batchData.length()=1
2023-12-04 18:20:56:114, BatchOctetMsg received a sample, counter=1
        topic       = pdxpBatchTopic, type = BatchOctetMsg
        dataLen     = 512
        dataBody    = [SSSS
```